

现代编程思想

特征与运算符重载

月兔公开课课程组

- 第六课：定义平衡二叉树
 - 定义一个更一般的二叉搜索树，允许存放任意类型的数据

```
1. enum Tree[T] {  
2.     Empty  
3.     Node(T, Tree[T], Tree[T])  
4. }  
5. // 我们需要一个比较函数来比较值的大小以了解顺序  
6. // 负数表示小于，0表示等于，正数表示大于  
7. fn[T] insert(tree : Tree[T], value : T, compare : (T, T) -> Int) -> Tree[T] { ... }  
8. fn[T] delete(tree : Tree[T], value : T, compare : (T, T) -> Int) -> Tree[T] { ... }
```

- 第八课：定义循环队列
 - 我们需要类型的默认值来初始化数组

```
1. fn[T] make(default : T) -> Queue[T] {  
2.     { array: Array::make(5, default), start: 0, end: 0, length: 0 }  
3. }
```

方法

- 数据结构上的操作通常被定义为：
 - `fn T::method(self : T, ...) -> ...`
 - `fn T::new() -> T`
- 允许方法调用语法、级联运算语法、链式调用：`x..f().g()`
- 我们需要某个类型支持一些方法：
 - 类型的比较：`fn T::compare(self : T, other : T) -> Int`
 - 类型的默认值：`fn T::default() -> T`
 - 类型的输出：`fn T::to_string(self : T) -> String`
 -

特征 Trait

- 特征的定义: `trait <特征名> [ : <超特征 + ...> ] { <方法类型签名>; ... }`

```
1. trait Compare : Eq {  
2.     compare(Self, Self) -> Int // Self代表实现该特征的类型  
3. }  
4. trait Default {  
5.     default() -> Self  
6. }
```

特征 Trait

- 我们可以在泛型的参数上添加特征的要求
 - 限制参数的类型: `<类型参数> : <特征1 + 特征2 + ...>`
 - 在函数中使用特征定义的方法: `<类型参数>::<方法名>` 或 `<变量名>.<方法名>`
- 例子:

```
1. // 类型参数 T 应当满足 Default 特征
2. fn[T : Default] make() -> Queue[T] {
3.   {
4.     // 我们可以利用特征中的方法, 返回类型为 Self, 即 T
5.     array: Array::make(5, T::default()),
6.     start: 0, end: 0, length: 0
7.   }
8. }
```

特征 Trait

- 特征可以尽早发现使用不存在方法的错误
- 错误例子：

```
1. struct BoxedInt {  
2.     value : Int  
3. }  
4.  
5. test {  
6.     let q : Queue[BoxedInt] = make() // 编译错误! BoxedInt 没有实现 Default  
7. }
```

特征 Trait

```
1. fn[T : Compare] insert(tree : Tree[T], value : T) -> Tree[T] {
2.     match tree {
3.         Empty => Node(value, Empty, Empty)
4.         Node(v, left, right) => {
5.             let result = value.compare(v) // 可以使用比较方法
6.             if result == 0 {
7.                 tree
8.             } else if result < 0 {
9.                 Node(v, insert(left, value), right)
10.            } else {
11.                Node(v, left, insert(right, value))
12.            }
13.        }
14.    }
15. }
```

特征实现

- 为类型实现某个特征: `impl <特征名> for <类型名> with <方法定义>`
- 特征实现需要实现特征中**所有**的方法
- 方法定义中的参数和返回值类型可以省略

```
1. struct BoxedInt { value : Int }
2.
3. impl Default for BoxedInt with default() { // 可以省略类型标注
4.     { value: Int::default() } // Int::default() 为 0
5. }
6.
7. test {
8.     let q : Queue[BoxedInt] = make()
9. }
```


特征实现

- 使用类型参数实现特征

```
1. struct Boxed[T] { value : T }  
2.  
3. impl[T : Default] Default for Boxed[T] with default() {  
4.   { value: T::default() }  
5. }  
6.  
7. test {  
8.   let q : Queue[Boxed[Int]] = make()  
9. }
```

- 特征可以提供默认实现

```
1. trait Eq {  
2.     equal(Self, Self) -> Bool  
3.     not_equal(Self, Self) -> Bool = _ // 这个方法有默认实现  
4. }  
5.  
6. impl Eq with not_equal(self, other) {  
7.     !self.equal(other) // 使用 equal 实现 not_equal  
8. }  
9.  
10. impl[T : Eq] Eq for Boxed[T] with equal(self, other) {  
11.     self.value.equal(other.value)  
12. }
```

- 手动实现会覆盖默认实现

```
1. impl[T : Eq] Eq for Boxed[T] with not_equal(self, other) {  
2.     self.value.not_equal(other.value)  
3. }
```

特征实现

- 所有方法都有默认实现的特征，也需要显式声明实现

```
1. trait Animal {  
2.     speak() -> String = _  
3. }  
4. impl Animal with speak() { "hi" }  
5.  
6. struct Dog {}  
7. impl Animal for Dog      // 显式声明实现
```

表：利用特征实现

- 一个表是键值对的集合
 - 对于每一个 键 存在一个对应 值
 - 例： `{ 0 -> "a", 5 -> "Hello", 7 -> "a" }`

```
1. type Map[K, V]
2.
3. // 创建表
4. fn[K, V] Map::make() -> Map[K, V] { ... }
5.
6. // 添加键值对，或更新键对应值
7. fn[K, V] Map::put(map : Map[K, V], key : K, value : V) -> Unit { ... }
8.
9. // 获取键对应值
10. fn[K, V] Map::get(map : Map[K, V], key : K) -> Option[V] { ... }
```

表：利用特征实现

- 表的简易实现
 - 利用数组+二元组存储键值对
 - 添加/更新时向数组末尾添加键值对
 - 查询时从数组开始遍历，找到键即返回
- 简易实现需要判断存储的键值对是否为搜索的键
 - 键应当满足 `Eq` 特征

```
1. fn [K : Eq, V] Map::put(map : Map[K, V], key : K, value : V) -> Unit { ... }  
2. fn [K : Eq, V] Map::get(map : Map[K, V], key : K) -> Option[V] { ... }
```

表：利用特征实现

- 我们以数组+二元组作为表

```
1. // 我们定义一个类型Map, 包含一个数组存储键值对
2. struct Map[K, V] {
3.   data : Array[(K, V)]
4. }
5.
6. fn[K, V] Map::make() -> Map[K, V] {
7.   { data: [] }
8. }
9.
10. fn[K : Eq, V] Map::put(self : Map[K, V], key : K, value : V) -> Unit {
11.   match self.data.search_by(fn(pair) { pair.0 == key }) {
12.     Some(index) => self.data[index] = (key, value)
13.     None => self.data.push((key, value))
14.   }
15. }
```

表：利用特征实现

- 我们以数组+二元组作为表

```
1. fn[K : Eq, V] Map::get(self : Map[K, V], key : K) -> Option[V] {  
2.     match self.data.search_by(fn(pair) { pair.0 == key }) {  
3.         Some(index) => Some(self.data[index].1)  
4.         None => None  
5.     }  
6. }
```

- 子特征可以在超特征上进行扩展: `trait Sub: Super1 + Super2 + ...`

```
1. trait Position {  
2.     pos(Self) -> (Int, Int)  
3. }  
4.  
5. trait Draw {  
6.     draw(Self, Int, Int) -> Unit  
7. }  
8.  
9. trait Object: Position + Draw {}
```

- 实现子特征必须先实现超特征

```
1. impl Position for T with pos(...) { ... }  
2. impl Draw for T with draw(...) { ... }  
3. impl Object for T
```


运算符重载

- 运算符和特征方法并没本质区别
- 月兔允许通过实现特定特征来重载运算符
 - 算术运算: `+` Add, `-` Sub, `*` Mul, `/` Div, `%` Mod, `-` Neg
 - 相等判断: `==` Eq
 - 比较运算: `<` Compare
 - 位运算: `|` BitOr, `&` BitAnd, `^` BitXOr, `<<` Shl, `>>` Shr
- 实现特定的运算符方法:

```
1. pub trait Add {
2.     add(Self, Self) -> Self
3. }
```

运算符重载

```
1. impl Eq for BoxedInt with equal(i : BoxedInt, j : BoxedInt) -> Bool {
2.   i.value == j.value
3. }
4.
5. impl Add for BoxedInt with add(i, j) {
6.   { value: i.value + j.value }
7. }
8.
9. test {
10.   inspect({ value: 10 } == { value: 100 }, content="false")
11.   inspect({ value: 10 } + { value: 100 }, content="{value: 110}")
12. }
```

下标运算符

- 下标运算符不通过特征实现，而是通过 `#alias` 注解
 - 取值运算符: `#alias("_[_]")`
 - 设值运算符: `#alias("_[_]=_")`

```
1. #alias("_[_]")
2. fn[K : Eq, V] Map::get(self : Map[K, V], key : K) -> Option[V] { ... }
3.
4. #alias("_[_]=_")
5. fn[K : Eq, V] Map::put(self : Map[K, V], key : K, value : V) -> Unit { ... }
6.
7. test {
8.   let map : Map[String, Int] = Map::make()
9.   map["hello"] = 5
10.  inspect(map["hello"], content="Some(5)")
11. }
```

- 特征对象允许在运行时实现多态
- 使用 `value as &TraitName` 语法将值转换为特征对象（类型擦除）

```
1. trait Animal {  
2.     speak(Self) -> String  
3. }  
4.  
5. struct Duck { name : String }  
6. impl Animal for Duck with speak(self) { "\{self.name}: quack!" }  
7.  
8. struct Fox { name : String }  
9. impl Animal for Fox with speak(self) { "\{self.name}: ring-ding-ding!" }  
10.  
11. test {  
12.     let animals : Array[&Animal] = [Duck::{ name: "Donald" }, Fox::{ name: "Nick" }]  
13.     for animal in animals {  
14.         println(animal.speak()) // Donald: quack! Nick: ring-ding-ding!  
15.     }  
16. }
```

特征对象和对象安全

- 并非所有的特征都可以创建特征对象
- 必须满足对象安全，所有的方法：
 - 第一个参数是 `Self`
 - 方法的类型中只能出现一个 `Self`

```
1. trait TraitA {  
2.     f(Self) -> Unit  
3. } // 对象安全，可以使用 &TraitA  
4.  
5. trait TraitB {  
6.     g(Self) -> Int  
7.     h() -> Self // 第一个参数不是 Self，返回值是 Self  
8. } // 不安全，不可以使用 &TraitB
```

特征对象和对象安全

- 反例：第一个参数不是 `Self`

```
1. trait Default {  
2.     default() -> Self // 第一个参数不是 Self, 不安全  
3. }  
4.  
5. test {  
6.     let x : &Default = ...  
7.     // 没有任何方式构造默认值  
8. }
```

特征对象和对象安全

- 反例：多个 Self 参数

```
1. trait Compare {  
2.     compare(Self, Self) -> Int // Self 出现了多次, 不安全  
3. }  
4.  
5. test {  
6.     let x : &Compare = 10  
7.     let y : &Compare = [1, 2, 3]  
8.     x.compare(y) // x 和 y 是不同类型, 不应该可以比较  
9. }
```

特征的自动派生

- 对于一些常用的特征，月兔可以自动生成实现
- 使用 `derive` 关键字在类型定义后声明

```
1. struct Point { x : Int; y : Int } derive(Eq, Show)
2.
3. enum Status { Active; Inactive } derive(Eq, Show, Default)
```

- 常见的可派生特征：
 - `Eq` : 判断相等性
 - `Compare` : 比较大小
 - `Show` : 转换为字符串
 - `Default` : 提供默认值

Eq 特征的自动派生

- Eq 特征用于判断相等性
- 结构体派生时，按字段定义顺序逐一比较
- 特征 T 的派生要求所有内部类型（结构体的字段，枚举的负载值类型）都实现了特征 T

```
1. struct Point { x : Int; y : Int } derive(Eq)
2.
3. test {
4.     let p1 = { x: 1, y: 2 }
5.     let p2 = { x: 1, y: 2 }
6.     let p3 = { x: 2, y: 2 }
7.     let p4 = { x: 1, y: 1 }
8.     inspect(p1 == p2, content="true") // p1.x == p2.x && p1.y == p2.y
9.     inspect(p1 == p3, content="false") // p1.x != p2.x
10.    inspect(p1 == p4, content="false") // p1.y != p2.y
11. }
```

Compare 特征的自动派生

- Compare 特征用于比较大小
- 枚举类型自动派生 Compare 会按照不同情形定义的先后顺序

```
1. enum Weekday {  
2.     Monday  
3.     Tuesday  
4.     Wednesday  
5.     Thursday  
6.     Friday  
7.     Saturday  
8.     Sunday  
9. } derive(Eq, Compare)  
10.  
11. test {  
12.     inspect(Monday < Tuesday, content="true")  
13.     inspect(Sunday < Monday, content="false")  
14.     inspect(Friday >= Friday, content="true")  
15. }
```

总结

- 本节课我们展示了：
 - 定义特征、类型约束、实现特征
 - 运算符重载
 - 特征对象
 - 特征的自动派生